Before DHCP: RARP on a Simulated Ethernet and BOOTP over UDP with a Relay Agent (RFC 903, RFC 951, RFC 1542)

for Computer Networks Laboratory

Name: Subham Bhat

Class: BCSE III

Roll: 002410501020

Group: A1

College: Jadavpur University

Contents

1 Introduction **3**

1.1 What Was Built 3

2 Packet Formats **4**

2.1 The RARP Frame 4

2.2 The BOOTP Message 5

3 System Architecture **7**

4 Workflows and Observations **8**

4.1 RARP on the Same LAN 8

4.2 RARP Cannot Cross a Router 8

4.3 BOOTP on the Same LAN 9

4.4 BOOTP Through the Relay Agent 9

4.5 Unknown Machines 10

5 Verification and Results **10**

6 Conclusion **12**

AI Acknowledgement **12**

1 Introduction

A **diskless workstation** boots from a ROM on its network card. The ROM knows the card's MAC address, but nothing else: no IP address, no subnet mask, no router, and no file to boot. Before it can load an operating system over the network, the machine must ask someone for this configuration. Two protocols solved this before DHCP existed, and DHCP is built directly on the second:

- **RARP** (Reverse ARP, **RFC 903**, 1984) reuses the ARP packet with new opcodes: "my MAC is X, what is my IP?". It runs directly on Ethernet (EtherType `0x8035`), so its broadcast **cannot cross a router**, and the answer is *only* an IP address.
- **BOOTP** (Bootstrap Protocol, **RFC 951**, 1985) runs over **UDP** (server port 67, client port 68). One reply carries the IP address, the boot server, the **boot file name**, and options such as mask, router and DNS (RFC 1497). A **relay agent** on the router (RFC 1542) forwards requests across subnets using the `giaddr` field.

In both protocols every address is fixed per MAC in a table typed in by an administrator; there are no pools and no leases. DHCP (1993) kept the BOOTP message, ports and relay mechanism and added dynamic allocation.

1.1 What Was Built

This module reproduces both protocols in pure Java 11 with no external libraries:

1. **RARP on a simulated Ethernet** with two LANs joined by a router: real 60-byte RARP frames (EtherType `0x8035`, opcodes 3 and 4), broadcast to every station, and a RARP server with an `/etc/ethers` table.
2. **BOOTP over real UDP**: the 300-byte message, a server with an `/etc/bootptab` table, a client with retries and back-off, and a **relay agent** that can be switched off.
3. **A step-by-step simulation** on `http://localhost:8085`: every frame and packet is animated on the network drawing and can be opened byte by byte.

Table 1: The lab network

Station	Where	Role
Boot server	LAN A, 10.0.1.5	RARP server (`/etc/ethers`) and BOOTP server (`/etc/bootptab`), also the TFTP boot server
Router	10.0.1.1 (LAN A), 10.0.2.1 (LAN B)	Joins the two LANs; optionally runs the BOOTP relay agent
WS-1	LAN A	Diskless, registered: 10.0.1.21
WS-4	LAN B	Diskless, registered: 10.0.2.24, but on the far side of the router
WS-3	LAN A	Diskless, not registered: the server stays silent

Table 2: Ports (BOOTP uses non-privileged copies of 67/68 so that it runs without administrator rights)

Port	Real port	Role
UDP 1067	67	BOOTP server
UDP 2067	67 (on the router)	Relay agent
UDP 1068	68	BOOTP client
HTTP 8085		Dashboard



Figure 1: The dashboard: the two-LAN network with the boot server, the router and the diskless workstations.

2 Packet Formats

2.1 The RARP Frame

RARP uses exactly the 28-byte ARP payload. Only the EtherType and the opcodes differ. Because the sender does not know its own IP address, the protocol fields are zero in the request, and the requester's MAC appears both as sender and as target hardware address.

Table 3: RARP frame (60 bytes on the wire, padded to the Ethernet minimum)

Bytes	Field	Value in a request
0–5	Destination MAC	`ff:ff:ff:ff:ff:ff` (broadcast)
6–11	Source MAC	the workstation's MAC
12–13	EtherType	`0x8035` = RARP (ARP is `0x0806`)
14–15, 16–17	HTYPE, PTYPE	1 = Ethernet, `0x0800` = IPv4
18, 19	HLEN, PLEN	6, 4
20–21	OPER	3 = reverse request, 4 = reverse reply
22–27, 28–31	SHA, SPA	own MAC, 0.0.0.0
32–37, 38–41	THA, TPA	own MAC, 0.0.0.0 (in the reply: TPA = the assigned IP)
42–59	padding	zeros

```
public static final int ETH_ARP = 0x0806, ETH_RARP = 0x8035;
public static final int REQUEST = 3, REPLY = 4;

public static byte[] frame(String dst, String src, int op, String sha, String spa, String
    tha, String tpa) {
    byte[] f = new byte[60];                                    // zero padding to 60 bytes
    System.arraycopy(Field.macBytes(dst), 0, f, 0, 6);
    System.arraycopy(Field.macBytes(src), 0, f, 6, 6);
    f[12] = (byte) (ETH_RARP >> 8); f[13] = (byte) ETH_RARP;  // EtherType 0x8035
    f[14] = 0; f[15] = 1;                                       // HTYPE 1 = Ethernet
    f[16] = 0x08; f[17] = 0x00;                                 // PTYPE 0x0800 = IPv4
    f[18] = 6; f[19] = 4;                                       // HLEN, PLEN
    f[20] = 0; f[21] = (byte) op;                               // 3 = request, 4 = reply
    System.arraycopy(Field.macBytes(sha), 0, f, 22, 6);
    System.arraycopy(Field.ipBytes(spa), 0, f, 28, 4);
    System.arraycopy(Field.macBytes(tha), 0, f, 32, 6);
    System.arraycopy(Field.ipBytes(tpa), 0, f, 38, 4);
    return f;
}
```

Listing 1: Building a RARP frame (`rarp/Rarp.java`).

2.2 The BOOTP Message

A BOOTP message is always 300 bytes: a 236-byte fixed part and a 64-byte vendor area. The vendor area begins with the **magic cookie** 99.130.83.99 (`63 82 53 63`), followed by options in type-length-value form. DHCP uses exactly the same layout and cookie and adds option 53 (message type).

Table 4: BOOTP message fields (RFC 951)

Offset	Field	Meaning
0	op	1 = BOOTREQUEST, 2 = BOOTREPLY
1, 2, 3	htype, hlen, hops	1 (Ethernet), 6, number of relay agents passed
4	xid (4)	Random transaction ID chosen by the client
8, 10	secs, flags	Seconds since the client started trying
12	ciaddr	Client IP, if it already knows it
16	yiaddr	“Your” IP: the address assigned by the server
20	siaddr	The boot (TFTP) server
24	giaddr	The relay agent’s address, if a relay forwarded the request
28	chaddr (16)	Client hardware address
44	sname (64)	Optional server host name
108	file (128)	Boot file name, e.g. `/tftpboot/sunos-ws1.img`
236	vend (64)	Magic cookie, then options: 1 mask, 3 router, 6 DNS, 12 host name, 255 end

```
public byte[] encode() {
    byte[] b = new byte[SIZE];                                  // SIZE = 300
    b[0] = (byte) op; b[1] = 1; b[2] = 6; b[3] = (byte) hops;
    b[4] = (byte) (xid >> 24); b[5] = (byte) (xid >> 16); b[6] = (byte) (xid >> 8); b[7] = (
    byte) xid;
    b[8] = (byte) (secs >> 8); b[9] = (byte) secs; b[10] = (byte) (flags >> 8); b[11] = (
    byte) flags;
    put(b, 12, Field.ipBytes(ciaddr)); put(b, 16, Field.ipBytes(yiaddr));
    put(b, 20, Field.ipBytes(siaddr)); put(b, 24, Field.ipBytes(giaddr));
    put(b, 28, Field.macBytes(chaddr));
    str(b, 44, 64, sname); str(b, 108, 128, file);
    put(b, 236, COOKIE);                                        // 99.130.83.99
    int p = 240;
    for (Map.Entry<Integer, byte[]> o : options.entrySet()) {     // type, length, value
        if (p + 2 + o.getValue().length >= SIZE) break;
        b[p++] = (byte) (int) o.getKey(); b[p++] = (byte) o.getValue().length;
        put(b, p, o.getValue()); p += o.getValue().length;
    }
    b[p] = (byte) OPT_END;
    return b;
}
```

Listing 2: Encoding the 300-byte BOOTP message (`bootp/Packet.java`).

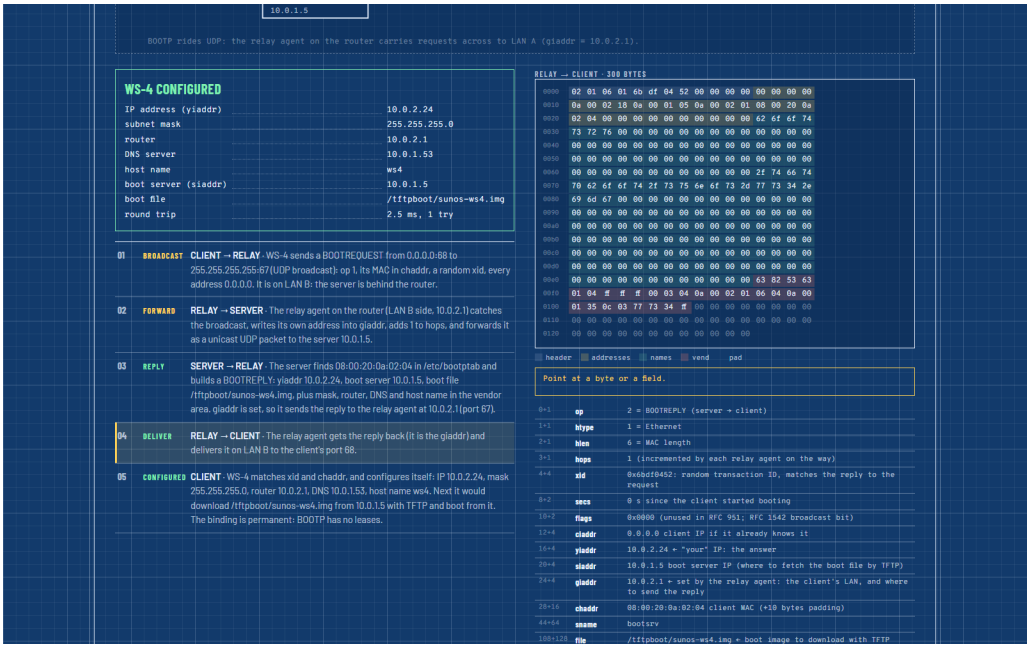


Figure 2: A BOOTREPLY opened byte by byte: yiaddr, siaddr, the boot file, the magic cookie and the options.

3 System Architecture

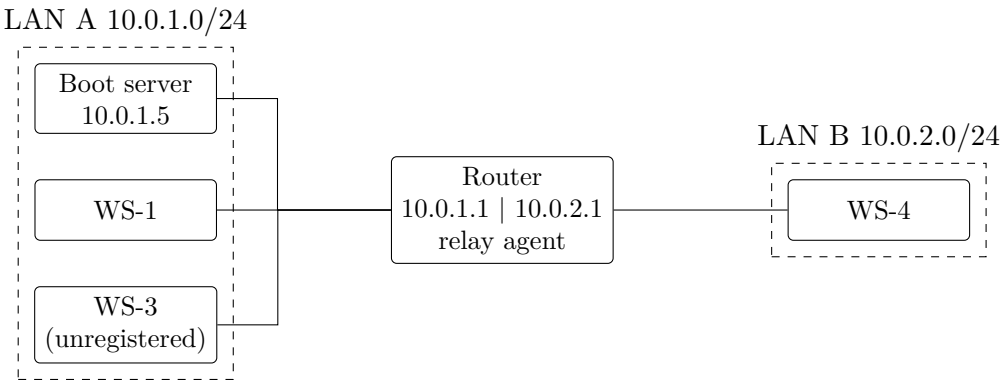


Figure 3: The simulated network: the boot server is on LAN A; WS-4 is on the far side of the router.

- `rarp/Lan.java` simulates the two Ethernet segments: a broadcast is delivered to every station on the same segment, and the router does not forward it.
- `rarp/Rarp.java` builds and dissects RARP frames.
- `bootp/Packet.java` encodes and decodes BOOTP messages; `bootp/Bootp.java` runs the server, the relay agent and the client on real UDP sockets.
- `dashboard/DashboardServer.java` serves the step-by-step simulation; every event carries the bytes and a decoded field list.

4 Workflows and Observations

4.1 RARP on the Same LAN

WS-1 broadcasts a RARP request (op 3). Every station on LAN A receives it; the router receives it too but does not forward it. The RARP server finds WS-1's MAC in `/etc/ethers` and sends a **unicast** reply (op 4) with TPA = 10.0.1.21. WS-1 now has an IP address, but still no mask, no router and no boot file; it would need further protocols to get them.

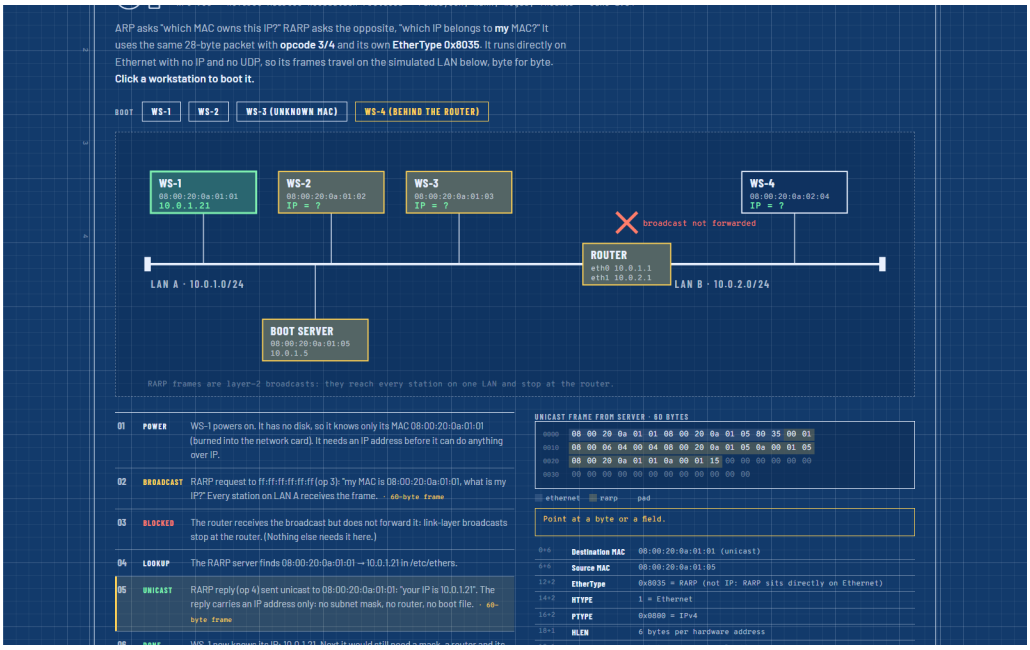


Figure 4: RARP on LAN A: broadcast request, unicast reply carrying only the IP address.

4.2 RARP Cannot Cross a Router

WS-4 on LAN B broadcasts the same request. The router receives it and drops it, because link-layer broadcasts stop at routers, and RARP has no IP header that could be routed. The server on LAN A never hears it. WS-4 retries after 1 s, 2 s and 3 s, then gives up, even though the server has its MAC in the table. With RARP, every LAN needs its own server.

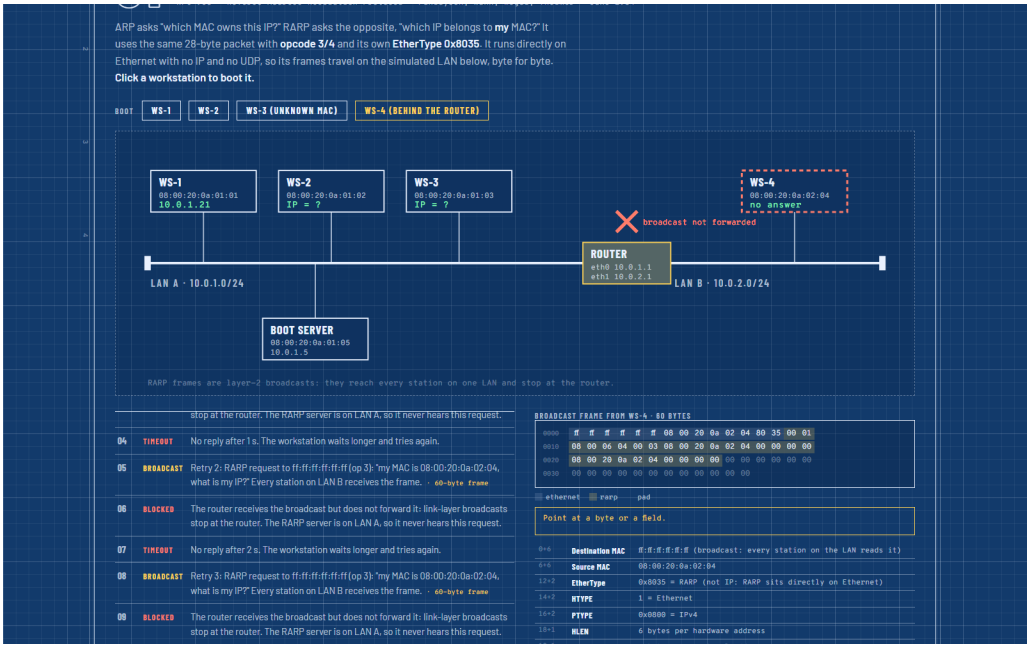


Figure 5: WS-4's RARP broadcast dies at the router; three retries, then failure.

4.3 BOOTP on the Same LAN

WS-1 sends a BOOTREQUEST from `0.0.0.0:68` to `255.255.255.255:67`: op 1, its MAC in `chaddr`, a random `xid`, all addresses zero. The server finds the MAC in `/etc/bootptab` and returns a BOOTREPLY with `yiaddr` 10.0.1.21, boot server 10.0.1.5, boot file `/tftpboot/sunos-ws1.img`, and mask, router 10.0.1.1, DNS 10.0.1.53 and host name `ws1` in the vendor area. WS-1 matches `xid` and `chaddr` and configures itself; next it would download the boot file with TFTP. The binding is permanent: BOOTP has no leases.

4.4 BOOTP Through the Relay Agent

The relay agent on the router listens for BOOTREQUESTs on LAN B, writes its own LAN-B address into `giaddr`, increments `hops`, and forwards the request as a **unicast** UDP packet to the server. Because `giaddr` is set, the server sends the reply back to the relay, which delivers it on LAN B to port 68. The same `giaddr` also tells the server which subnet the client is on.

```
Packet p = Packet.decode(buf, dp.getLength());
if (p.op == Packet.BOOTREQUEST) {
    if (!relayOn) { step(p.xid, "RELAY", "-", "dropped", "no relay agent: the broadcast dies
     at the router", null); continue; }
    if (p.hops >= 4) { step(p.xid, "RELAY", "-", "dropped", "hops limit reached, discarded",
     null); continue; }
    p.hops++;
    if (p.giaddr.equals("0.0.0.0")) p.giaddr = Topology.ROUTER_B_IP;    // 10.0.2.1
    byte[] out = p.encode();
    relay.send(new DatagramPacket(out, out.length, LO, SERVER_PORT));   // unicast to the
    server
} else {
    byte[] out = p.encode();                                            // BOOTREPLY coming
    back
    relay.send(new DatagramPacket(out, out.length, LO, CLIENT_PORT));  // deliver on LAN B,
    port 68
}
```

Listing 3: The relay agent (`bootp/Bootp.java`).

WS-4 receives 10.0.2.24 with router 10.0.2.1. With the relay agent switched off on the dashboard, the same request is dropped at the router, and WS-4 fails after three tries, exactly like RARP.

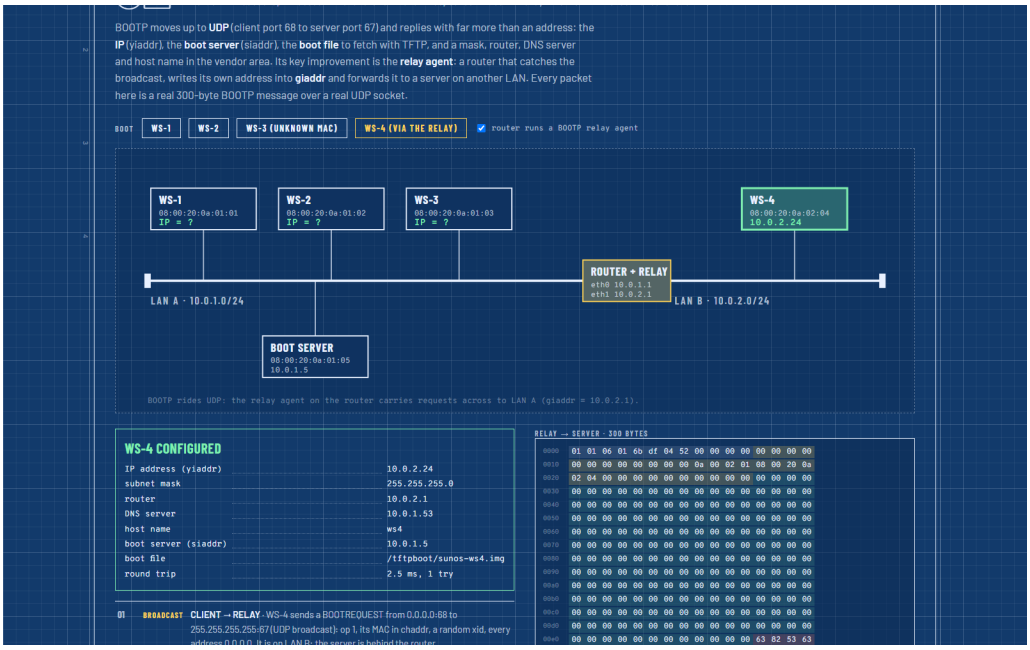


Figure 6: BOOTP through the relay: the forwarded request carries giaddr = 10.0.2.1 and hops = 1.

4.5 Unknown Machines

WS-3 is not in either table. Both servers stay silent, because neither protocol has a pool of spare addresses; the client retries three times and gives up. Registering WS-3 from the dashboard's table and booting it again succeeds. This manual table is the main reason DHCP was introduced.

5 Verification and Results

```
cd RARP-BOOTP
run.bat                    # RARP LAN, BOOTP server + relay, dashboard (http://localhost:8085)
run.bat --verify           # 19 automated checks
python interop_test.py     # a BOOTREQUEST built by hand in Python against our server and
    relay (7 checks)
```

Listing 4: Running the lab, the checks and the Python client.

Table 5: Verification checks (all 19 passed)

RARP frame is padded to the 60-byte Ethernet minimum; EtherType `0x8035`, not `0x0806`; payload is 28 bytes like ARP
Op 3 = reverse request with SHA = THA = own MAC and SPA = TPA = 0.0.0.0
Request is a broadcast heard by every station on LAN A; reply is unicast (op 4) with TPA = 10.0.1.21
Unregistered MAC: the server stays silent, three tries, then failure
WS-4 on LAN B fails with RARP: the router does not forward the broadcast
BOOTP message is always 300 bytes; round trip keeps xid, chaddr, yiaddr and file
Vendor area starts with the magic cookie 99.130.83.99; the subnet-mask option decodes
WS-1 gets IP 10.0.1.21 with mask, router, DNS, host name, boot server and boot file
WS-4 succeeds through the relay agent, which set giaddr = 10.0.2.1 and hops = 1
Relay agent switched off: WS-4 fails, just like RARP

A BOOTREQUEST built byte by byte in Python, independently of our code, receives a correct BOOTREPLY both directly and through the relay agent (all 7 interoperation checks pass). A screen recording of the full simulation is in `docs/video`.

Table 6: RARP, BOOTP and DHCP compared

	RARP	**BOOTP**	**DHCP**
Layer	Link (EtherType 0x8035)	UDP 67/68	UDP 67/68
Crosses routers	No	Yes, with a relay agent	Yes, with a relay agent
Gives	IP address only	IP, mask, router, DNS, boot file	All of BOOTP plus many options
Allocation	Static table	Static table	Pools, leases, renewal
Unknown client	No answer	No answer	Gets an address from the pool

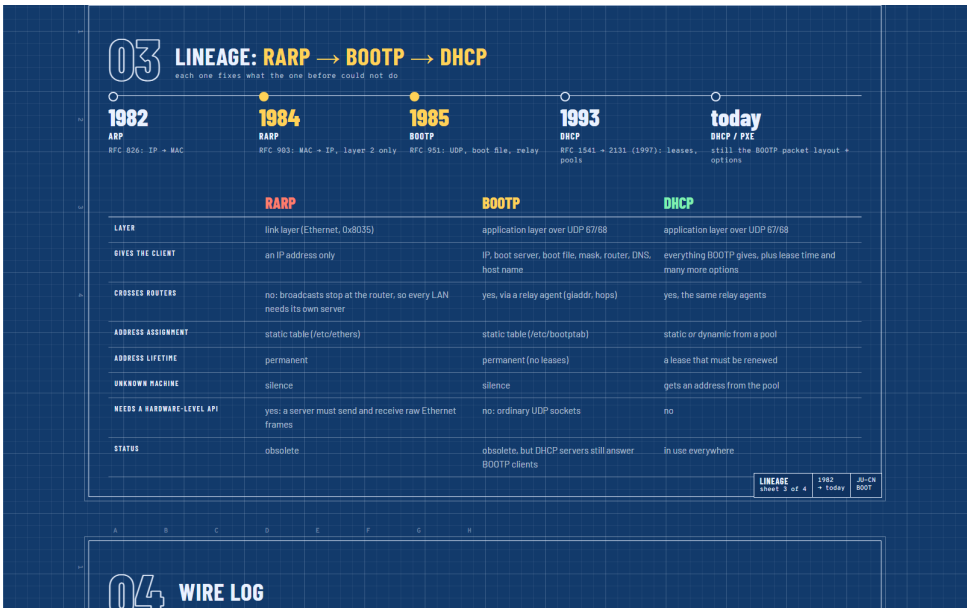


Figure 7: The lineage RARP → BOOTP → DHCP.

6 Conclusion

1. **The layer decides the reach.** RARP lives directly on Ethernet, so its broadcast stops at the first router; BOOTP's move to UDP/IP made forwarding possible.
2. **The relay agent is the key idea.** By catching the broadcast, writing its own address into `giaddr` and forwarding it as unicast, one central server can configure many subnets. DHCP uses the same mechanism unchanged.
3. **Static tables do not scale.** Both protocols ignore unregistered machines and never reclaim addresses; DHCP kept the BOOTP message and added address pools and leases to solve exactly this.

AI Acknowledgement

Artificial Intelligence tools were used as an aid during this project. Specifically, AI assistance was used for:

- **Protocol cross-checking:** comparing message formats, reply codes, state machines and byte offsets against the RFCs cited in this report.
- **Code review and testing:** suggesting edge cases for the automated verification suite and reviewing concurrency and error handling.
- **Writing and visualization:** drafting, structuring and refining the explanations, tables, diagrams and LaTeX source of this report.

All protocol implementations, Java source code, experiments, measurements and final conclusions were run, reviewed and verified by the author, who takes full responsibility for the content of this report.